

Task Management System.

Factory Method and Strategy, built in pure Java.

 Factory Method  Strategy  State

A task tracker for software teams, designed around two classic patterns and a zero-dependency Java core.

Abdul Rahman Malak
Mouhammad Houjeirat

What this presentation covers.



01

Introduction and problem definition

~3 min



02

The design patterns: Factory Method, Strategy, State

~5 min



03

Architecture and implementation

~4 min



04

Live demonstration

~3 min



05

Results, evaluation and conclusion

~2 min

Roughly sixteen minutes of talk and demo, then questions. We each take questions on the parts we led.



Introduction and problem definition.

The domain we chose, the objectives we set, and the two problems that drove every pattern choice.

Why a task management system?

Software teams track many kinds of work. How that work gets created and ordered is where a tidy codebase usually starts to sprawl.



The domain

A tracker for a software team: bug, feature, and documentation tasks, each moving through one shared lifecycle from open to done.



The motivation

Two recurring causes of code sprawl in this kind of system line up cleanly with two patterns from the course.



Our objective

Support new task types and new orderings, and let them be added without editing code that already works.



In one line

One engine, three ways to run it, sixteen Java files, zero external dependencies.

What the system must do.



FUNCTIONAL REQUIREMENTS

- › Create bug, feature, and documentation tasks.
- › Order the list by priority, deadline, or severity.
- › Move tasks through a validated lifecycle.
- › Filter by status and summarise the list.



NON-FUNCTIONAL REQUIREMENTS

- › Extensible: new types or orderings, no edits to old code.
- › Pure Java 8+, zero external dependencies.
- › Runs from the terminal: tests, console, and a GUI.
- › Explainable: the design has to survive a viva.

Two requirements drive the whole design: tasks are created in different ways, and the same list is ordered in different ways.

Tasks have different creation logic.

Each task type has different fields and different defaults. Building them by hand ties every caller to the concrete classes.

```
new BugTask(title, priority, "MEDIUM", "");           // severity + steps
new FeatureTask(title, priority, 8, 5);               // effort + value
new DocumentationTask(title, priority, "API", "Developers");
```



The flaw

Every entry point imports and names BugTask, FeatureTask, and DocumentationTask directly.



The cost

Adding a fourth type means hunting down and editing every place that builds a task.



What we need

We need to make a task from a type identifier, so the caller never names a concrete class.

Ordering depends on the moment.

The same task list is sorted differently at different times. The list never changes; only the rule does.

```
switch (sortMode) {  
    case DEADLINE: /* sort by due date    */ break;  
    case SEVERITY: /* bugs first, by rank */ break;  
    case URGENT:   /* sort by priority    */ break;  
}
```



The flaw

One switch statement inside the manager grows to carry every ordering rule.



The cost

Each new context edits tested code, which breaks the Open/Closed principle.



What we need

The ordering should swap at runtime, and new orderings should drop in without touching the manager.



The design patterns.

Two patterns chosen to match the two problems, plus a third that the task lifecycle gave us at no extra cost.

 **Factory Method** creational  **Strategy** behavioural  **State** behavioural

Matching patterns to problems.

We did not start from the patterns. We started from the two problems, and each one maps cleanly to a single pattern.



Problem 1 · different creation logic



Factory Method
Creational pattern

Decouples the caller from the concrete task classes.



Problem 2 · context-dependent ordering



Strategy
Behavioural pattern

Makes each ordering a swappable object, chosen at runtime.



Bonus · the task lifecycle



State (in an enum)
Behavioural pattern

Each status declares its own legal transitions.

Factory Method, and why it fits.



Definition

Define a method for making an object, and let each subclass decide which concrete class it produces.



Why here

Every task type has its own construction rules and defaults. The caller should not have to know them.



Advantage

A new task type is a new class. Code that creates tasks never changes when a type is added.



Real world

A document app that opens PDF, DOCX, and TXT files through one Open action, without naming a concrete reader.

How we built it.

The manager keeps a registry of factories keyed by a string. The caller asks for a type by name and never sees a concrete class.

```
manager.createTask("BUG", "Login broken", "OAuth callback fails", 8);  
manager.createTask("FEATURE", "Dark mode", "Top user request", 5);
```



An abstract class, not an interface

Each subclass owns its defaults: a bug is MEDIUM severity, a feature is 8h of effort at value 5, a doc targets developers.



A template method on the base

createTaskWithDeadline wraps createTask and attaches a deadline once. Written once, reused by all three factories.

Strategy, and why it fits.



Definition

Define a family of interchangeable algorithms, wrap each one, and select between them at runtime.



Why here

The same task list must be ordered differently in different moments, while the app is running.



Advantage

Add or switch an ordering without touching the code that holds and manages the list.



Real world

Like picking a route in a maps app: fastest, shortest, or no tolls, over the same map.

How we built it.

Each ordering is a swappable object behind one interface. Changing the sort is a single setter call; the manager never learns which one is active.

```
manager.setPriorityStrategy(new SeverityFirstStrategy());  
// the next call to getPrioritizedTasks() returns the new order
```



An interface, not a raw Comparator

PriorityStrategy names the role. SeverityFirst also lifts bugs to the top before sorting the rest, which a pairwise Comparator handles awkwardly.



A setter, not a constructor argument

Users change the sort mid-session. A setter avoids rebuilding the manager and copying every task into a fresh instance.

The same list, reordered live.

SEN3006 Task Manager

Demo Help

Create Task

Type: Title: Priority:

Description: + Add Task

Sort by: Filter:

ID	Type	Title	Priority	Status	Deadline
1	BUG	Fix payment bug	5	OPEN	2026-05-01
5	FEATURE	Export CSV	4	OPEN	2026-05-10
4	DOCUMENTATION	Setup guide	3	OPEN	2026-04-20
2	FEATURE	Add search	2	OPEN	2026-06-15
3	BUG	UI glitch	1	OPEN	—

Selected Task Actions

→ IN_PROGRESS → REVIEW → DONE Block Unblock (→ OPEN) Delete

5 tasks | 5 OPEN | Strategy: UrgentFirstStrategy

Urgent First sort: priority 5 down to 1.

SEN3006 Task Manager

Demo Help

Create Task

Type: Title: Priority:

Description: + Add Task

Sort by: Filter:

ID	Type	Title	Priority	Status	Deadline
1	BUG	Fix payment bug	5	OPEN	2026-05-01
3	BUG	UI glitch	1	OPEN	—
5	FEATURE	Export CSV	4	OPEN	2026-05-10
4	DOCUMENTATION	Setup guide	3	OPEN	2026-04-20
2	FEATURE	Add search	2	OPEN	2026-06-15

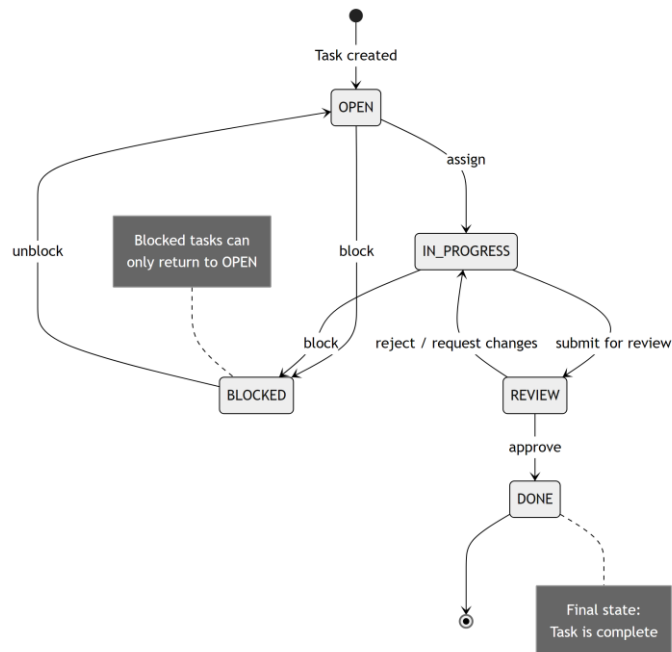
Selected Task Actions

→ IN_PROGRESS → REVIEW → DONE Block Unblock (→ OPEN) Delete

5 tasks | 5 OPEN | Strategy: SeverityFirstStrategy

Severity First sort: both bugs rise to the top.

The lifecycle is a state machine.



TaskStatus owns its transitions

Each enum constant declares its legal next states through `canTransitionTo()`.

The base class enforces them

`AbstractTask.setStatus()` checks that table and throws on an illegal move.

No shortcuts

The engine refuses to jump **OPEN** straight to **DONE**; the cycle has to be walked.

State, for zero extra files

The whole pattern lives in one enum, with no extra classes.

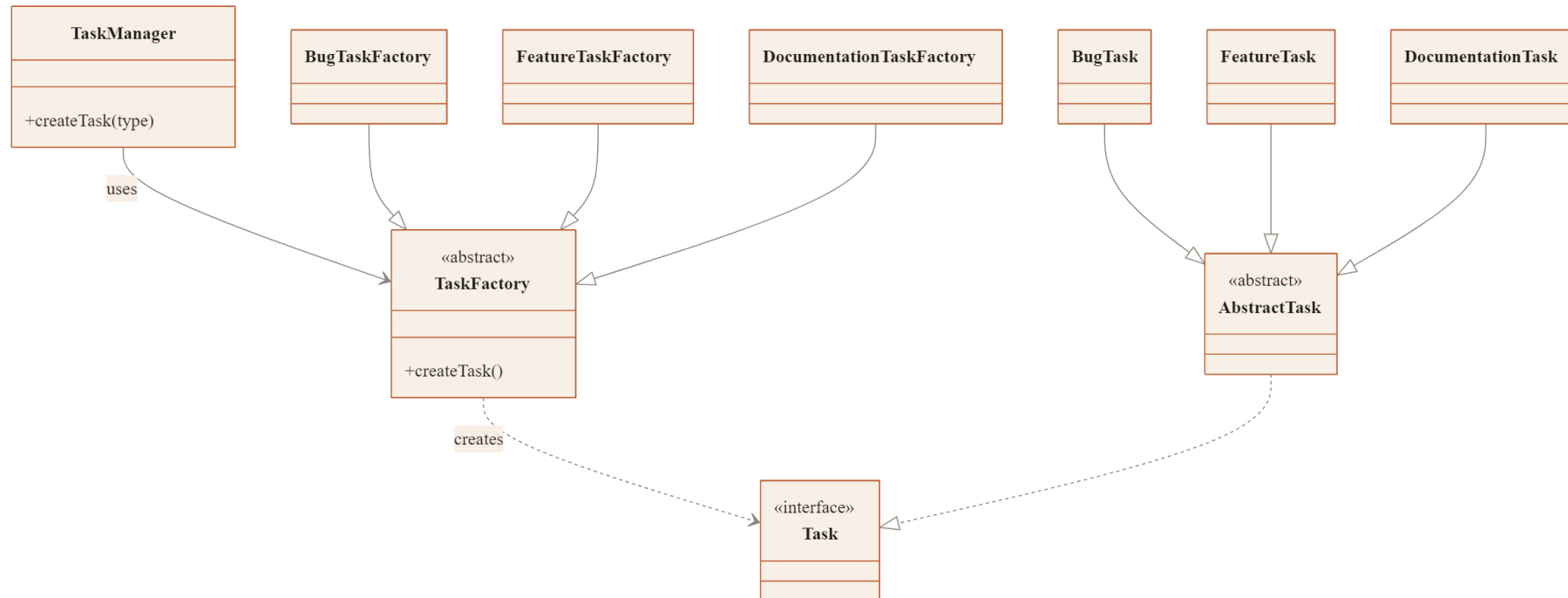


Architecture and implementation.

How the design looks as classes, how one request flows through it, and how it satisfies all five SOLID principles.

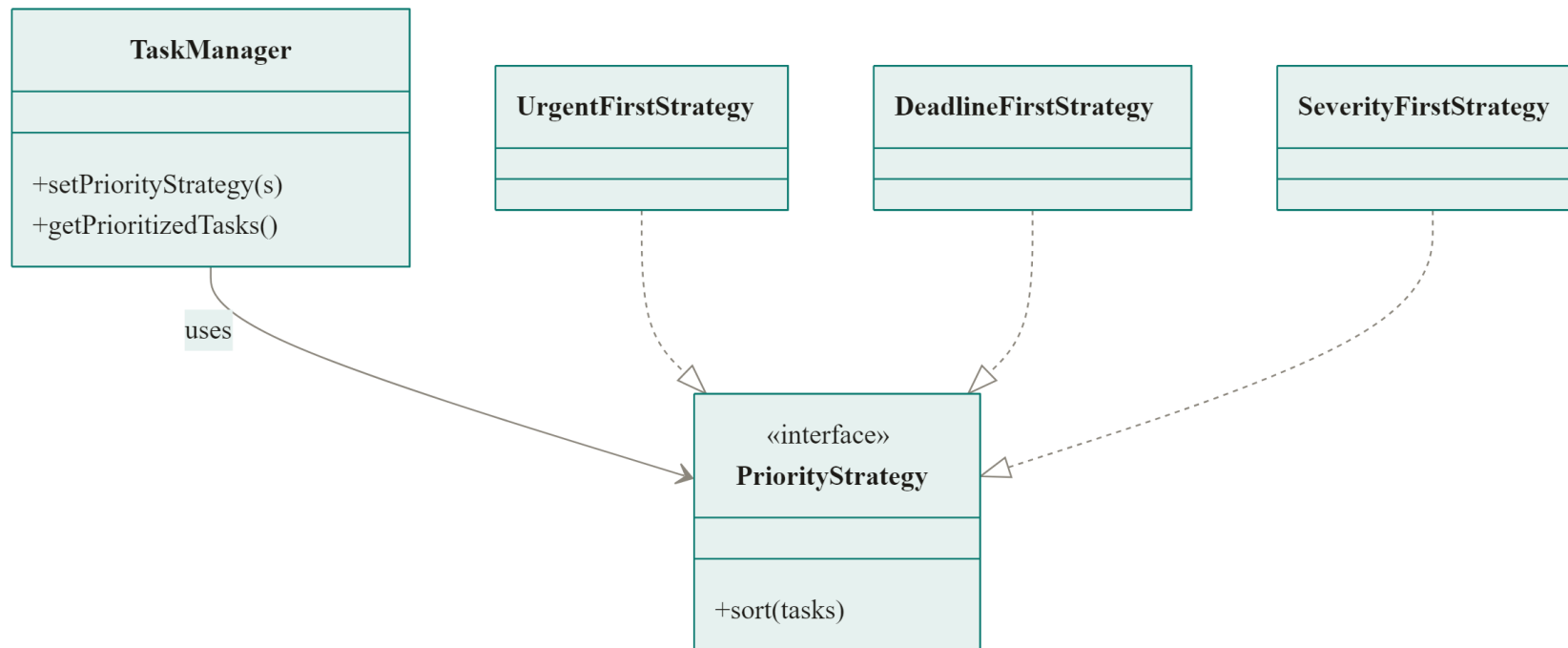
Class view — the Factory Method side.

TaskManager talks only to the abstract TaskFactory, which creates a Task. Each concrete factory and each concrete task slots in underneath.



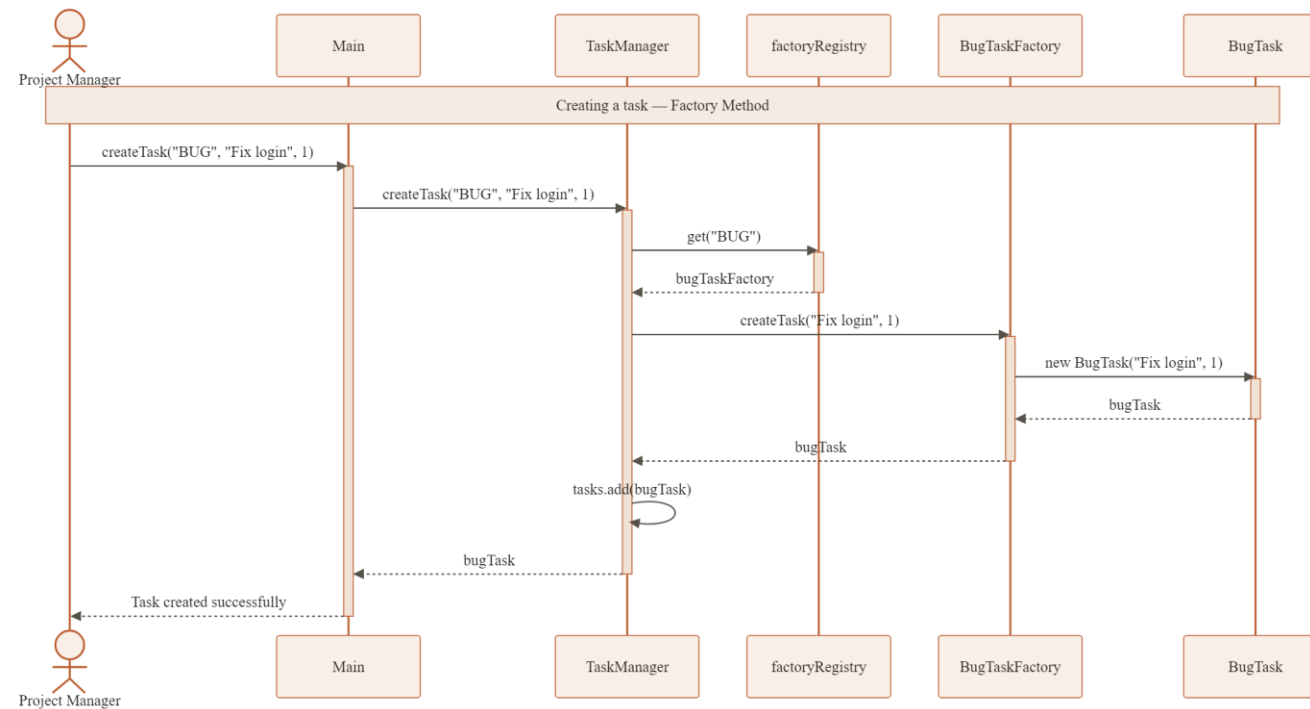
Class view — the Strategy side.

TaskManager holds one PriorityStrategy and can swap it at runtime. Each ordering is an interchangeable implementation of the same interface.



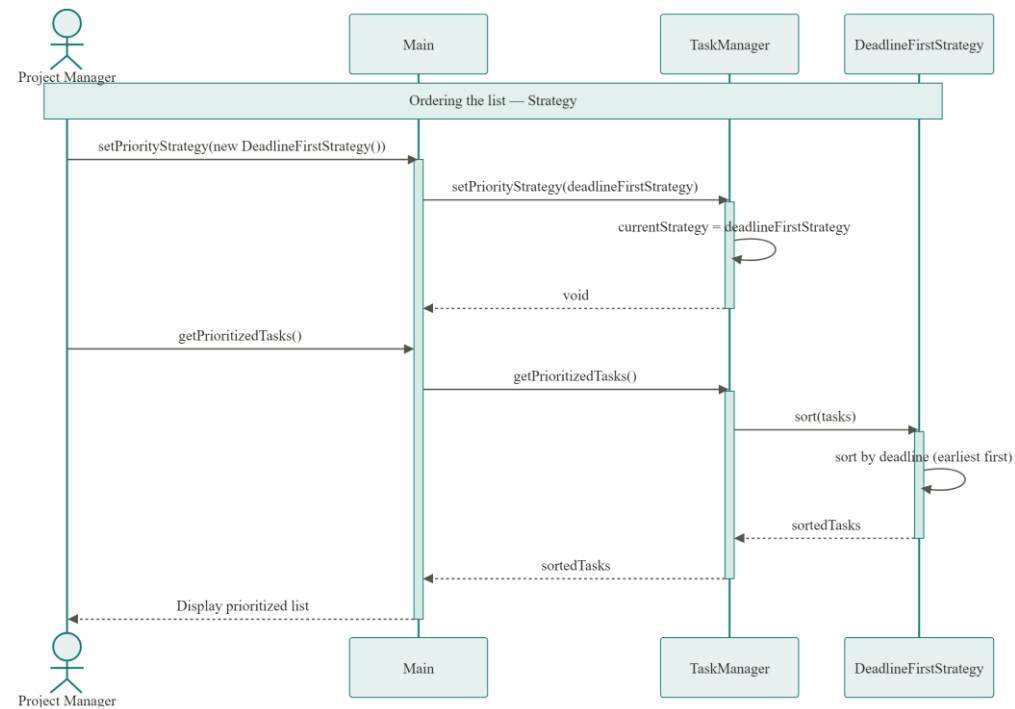
Creating a task, step by step.

The caller passes a type string; the manager looks up the registered factory and returns a Task. No concrete class is ever named by the caller.



Ordering the list, step by step.

Set a strategy, then ask for the prioritized list. The manager delegates the sort to whichever strategy is currently set.



Sixteen files, three layers.

No package declarations, only java.util and java.time imported. The columns mirror the three layers of the class diagram.



PRODUCT LAYER

- › Task (interface)
- › AbstractTask (base)
- › BugTask, FeatureTask
- › DocumentationTask
- › TaskStatus (enum)



PATTERN LAYER

- › TaskFactory (abstract)
- › Bug / Feature / Doc factories
- › PriorityStrategy (interface)
- › UrgentFirst, DeadlineFirst
- › SeverityFirst



COORDINATION

- › TaskManager (coordinator)
- › Main (automated tests)
- › TaskManagementApp (console)
- › gui.TaskManagerGUI (Swing)

All three entry points drive the same engine through its public API. 2 interfaces · 1 enum · 1 abstract · 12 concrete classes.

How the design meets SOLID.



S · Single responsibility

Factories build, strategies sort, the manager coordinates. One reason to change each.



O · Open / Closed

A new task type or sort is a new file plus one line. Zero edits to existing classes.



L · Liskov substitution

Every factory works through the abstract reference; every strategy through the interface.



I · Interface segregation

PriorityStrategy declares one method; the Task interface stays minimal.



D · Dependency inversion

The manager's fields are all interfaces and abstract types, never concretes.



The throughline

Abstractions sit in the middle of the system; concrete classes live at the edges.



Live demonstration.

The compiled system, run three ways: the automated test suite, the interactive console, and the Swing GUI.

Demo 1 — the terminal.

Open a terminal in the project root folder, then paste the command for each step.

```
java -cp bin Main

$ java -cp bin Main

##### Task Management System #####

TEST 1  Factory Method Pattern
[PASS] correct types, no concretes
TEST 2  Strategy Pattern
[PASS] one list, three orderings
TEST 3  Lifecycle / State machine
[PASS] illegal moves are blocked
TEST 4-6 edge cases ..... 6 / 6

##### ALL TESTS PASSED #####
```

STEP 1 · BUILD ONCE

```
javac -d bin src/main/java/*.java src/main/java/gui/*.java
```

STEP 2 · RUN THE AUTOMATED TESTS

```
java -cp bin Main
```

STEP 3 · OPEN THE INTERACTIVE CONSOLE

```
java -cp bin TaskManagementApp
```

Inside the console menu:

1 create a task (Factory) · 4 change the sort (Strategy) · 3 view sorted · 5 change status (State)

Demo 2 — the graphical interface.

From the same project root, one command launches the whole interface — the rest is clicks.

ID	Type	Title	Priority	Status	Deadline
1	BUG	Fix payment bug	5	OPEN	2026-05-01
5	FEATURE	Export CSV	4	OPEN	2026-05-10
4	DOCUMENTATION	Setup guide	3	OPEN	2026-04-20
2	FEATURE	Add search	2	OPEN	2026-06-15
3	BUG	UI glitch	1	OPEN	—

PASTE THIS TO LAUNCH THE GUI

```
java -jar TaskManagerGUI.jar
```



Load the demo data

Demo ▶ Load Strategy Demo seeds the table with tasks.



Strategy, live

Sort by: Urgent, Deadline, Severity — the table reorders, no recompile.



Factory, live

Add a task; the Type field picks which factory builds it.



State machine

Mark OPEN ▶ DONE: refused. OPEN ▶ IN_PROGRESS: allowed.



Results, evaluation and conclusion.

What the patterns delivered in practice, the honest limits of this build, and the main lesson.

New types and orderings, without editing existing code.



EXTENSION, IN NUMBERS

- › New task type: 2 new files, 1 line of registration, 0 edits.
- › New ordering: 1 new file, 1 setter call, 0 edits.
- › Test 5 in Main asserts this Open/Closed property at runtime.



WHAT WORKED IN PRACTICE

- › Documentation type was added late, with zero edits to existing code.
- › The GUI dropdown proves the Strategy swap, live.
- › The lifecycle enum makes every illegal transition unreachable.
- › Main doubles as the test harness, with no JUnit needed.

An honest look at the design.



Limitations

All data is in memory; nothing persists. Single user, no concurrency. Validation throws on the first error rather than collecting them all.



Alternatives we weighed

Abstract Factory: rejected, there is only one product family. A raw Comparator: loses the named role and the stateful sort.



Future work

Persistence via Repository, tags via Specification, async dispatch, and a small REST layer over the same engine.

Every limitation above is a scoping choice, not a structural one. The engine is decoupled enough that each item is an addition, not a rewrite.

What we set out to do, and what we learned.



What we achieved

A working task system where new types and new orderings cost no edits to existing, tested code.



The patterns earned their place

Each was chosen to solve a concrete problem from section one, not to satisfy a checklist.



The lesson

Naming a role, a factory or a strategy, makes the codebase explain itself to the next reader.



The bigger point

Design patterns are a shared vocabulary that lets a team extend software without breaking what already works.

Thank you.

Questions welcome.

We can speak to any part: the problem, the patterns, the architecture, the code, or the demo.

 Factory Method

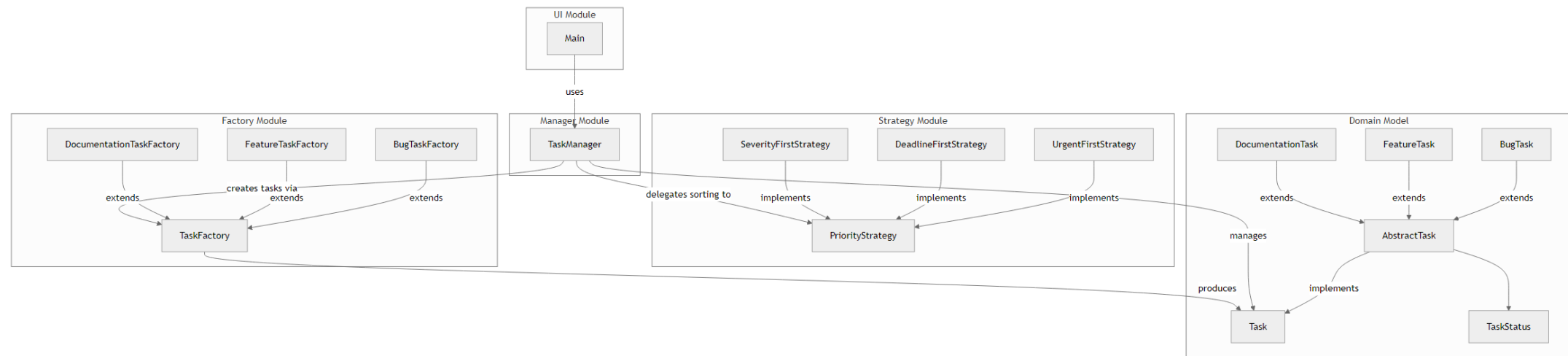
 Strategy

 State

SEN3006 · SOFTWARE ARCHITECTURE

Component view.

The module structure: the UI depends on the manager, which delegates to the factory and strategy modules over the shared domain model.



Each box is a module; every arrow is a compile-time dependency pointing inward toward the manager and the abstractions.

Deployment view.

One JVM, one shared engine, three entry points. No network, no database, no external services.

